

Universidade de São Paulo
Instituto de Ciências Matemáticas e de Computação
Computação de Alto Desempenho — SSC0903

Primeiro Trabalho Prático (TB1)

Simulação paralela da propagação direcional de incêndio com
zonas de contenção

Integrantes

Felipe de Castro Azambuja — 14675437
Gabriel de Andrade Abreu — 14571362
João Pedro Viguini Tolentino Taufner Correa — 14675503
Matheus Paiva Angarola — 12560982
Pietra Gullo Salgado Chaves — 14603822

São Carlos, 2026

Sumário

1	Introdução	1
1.1	Objetivos	1
1.2	Resumo da abordagem	1
2	Solução Sequencial	1
2.1	Estruturas e preparação	1
2.2	Passo da simulação	2
2.3	Ativação das zonas de contenção	2
2.4	Cálculo de estatísticas e <i>checksum</i>	2
3	Solução Paralela com OpenMP	2
3.1	Região paralela persistente	3
3.2	Paralelização da ativação das zonas	3
3.3	Paralelização da atualização da matriz	3
3.4	Reduções para contadores	3
3.5	Uso de SIMD e diagnóstico do compilador	3
3.6	Cláusulas de escopo e <code>default(none)</code>	4
3.7	Troca segura das matrizes e condição de parada	4
3.8	Diagrama da solução	4
3.9	Comparação de <i>schedules</i>	4
4	Validação	5
4.1	Metodologia de comparação	5
4.2	Casos de teste	5
4.3	Determinismo em relação ao número de threads	5
5	Ambiente e Metodologia Experimental	5
5.1	Hardware e software	6
5.2	Parâmetros das entradas de teste	6
5.3	Metodologia de medição	6
6	Resultados	7
6.1	Visualização do comportamento das entradas	7
6.2	Tempo, <i>speedup</i> e eficiência	9
6.3	Comparação de <i>schedules</i>	10
7	Análise dos Resultados	11
7.1	Escalabilidade observada	11
7.2	Gargalos identificados	11
7.3	Justificativa do <i>schedule</i> escolhido	11
8	Conclusão	11

1. Introdução

Este trabalho implementa uma simulação determinística da propagação de incêndio em uma matriz retangular. Foram desenvolvidas uma versão sequencial em C e uma versão paralela com OpenMP. Ambas consideram cobertura, umidade, vento, focos iniciais, tempo de queima e zonas de contenção. O objetivo da paralelização é reduzir o tempo do núcleo da simulação sem modificar o resultado.

As duas versões compartilham as mesmas regras e estruturas. A comparação experimental verifica a correção, o tempo de execução, o *speedup*, a eficiência e o efeito de diferentes *schedules*.

1.1 Objetivos

Os objetivos são implementar corretamente as regras da simulação, garantir resultados idênticos entre as duas versões e avaliar a paralelização por meio de tempo, *speedup*, eficiência e comparação de *schedules*.

1.2 Resumo da abordagem

A mesma representação linear e a mesma lógica de atualização são usadas nos dois programas. A versão sequencial serve como referência, enquanto a versão OpenMP divide os laços sobre as células entre as threads e sincroniza as estatísticas e a troca de matrizes ao final de cada passo.

2. Solução Sequencial

Esta seção apresenta a organização da versão sequencial, usada como referência de correção. São descritos o armazenamento das matrizes, as regras de propagação, a ativação das contenções e o cálculo das estatísticas finais.

2.1 Estruturas e preparação

As matrizes são armazenadas linearmente pelo índice

$$indice = linha \times C + coluna.$$

Para cada célula são mantidos cobertura, umidade, estado e tempo de queima. A simulação utiliza pares de vetores para estado e tempo: `estado_atual/proximo_estado` e `tempo_atual/proximo_tempo`. Essa organização permite calcular o próximo passo exclusivamente a partir do passo atual.

A cobertura e a umidade são geradas sequencialmente, em ordem linear, com `rand_r`. Em seguida são aplicados os focos iniciais e construído o vetor `ativacao`.

Quando zonas se sobrepõem, o vetor guarda o menor passo de ativação.

2.2 Passo da simulação

Em cada passo, o programa:

1. ativa as contenções programadas sobre células intactas;
2. calcula o próximo estado e o próximo tempo de todas as células;
3. conta novas ignições e células que continuarão em chamas;
4. troca os pares de matrizes;
5. encerra caso não existam células em chamas ou tenham sido executados P passos.

Para uma célula intacta, são examinados os oito vizinhos de Moore. Cada vizinho em chamas contribui com peso básico 10, se ortogonal, ou 7, se diagonal. Considerando o alinhamento A com o vento, o peso é

$$P_v = \max(1, P_{basico} + intensidade \times A).$$

O potencial de ignição é calculado com aritmética inteira:

$$I = \left\lfloor \frac{S \times fator_{combustivel} \times (100 - umidade)}{100} \right\rfloor.$$

A célula entra em chamas quando $I \geq LIMIAR$. O tempo de queima é 2 para vegetação rasteira e 4 para floresta.

2.3 Ativação das zonas de contenção

No início do passo p , células intactas cujo valor em `ativacao` seja igual a p passam ao estado de contenção. Células não combustíveis, em chamas, queimadas ou já protegidas não são alteradas. Uma zona atua somente no passo programado.

2.4 Cálculo de estatísticas e *checksum*

Ao final são calculados os estados, o total e o pico de ignições, os percentuais queimado e protegido e o *checksum*. O *checksum* é sequencial e fica fora do trecho cronometrado.

3. Solução Paralela com OpenMP

Esta seção descreve como o núcleo sequencial foi paralelizado com OpenMP sem alterar as regras da simulação. Também são discutidos o compartilhamento de dados, as reduções, a sincronização e os últimos critérios de implementação.

3.1 Região paralela persistente

A versão paralela utiliza uma única região persistente com T threads e `default(none)`. A preparação da entrada e a geração pseudoaleatória permanecem sequenciais, garantindo a mesma floresta da versão de referência.

3.2 Paralelização da ativação das zonas

A varredura do mapa de ativação é distribuída com `omp for` e `collapse(2)`. Cada iteração trata uma célula diferente, portanto as threads não escrevem na mesma posição.

3.3 Paralelização da atualização da matriz

Dentro de cada passo, três operações sobre a matriz são distribuídas com `omp for`: ativação das contenções, atualização das células e cálculo das estatísticas. Como cada iteração de atualização escreve em uma posição distinta dos vetores de próximo estado e próximo tempo, não há disputa de escrita entre threads. A contagem das novas ignições e das células em chamas combina `omp for simd` com redução. Após o último passo, a mesma equipe persistente distribui a contagem dos cinco estados finais com outra construção `omp for simd`.

3.4 Reduções para contadores

Reduções OpenMP calculam as novas ignições e as células em chamas em cada passo. Uma segunda redução, ainda dentro da região persistente, acumula as quantidades finais de células não combustíveis, intactas, em chamas, queimadas e de contenção. Isso dispensa `critical` e `atomic` nos laços. Uma única thread atualiza o pico, acumula o total de ignições e troca os ponteiros com `single`. As barreiras implícitas asseguram que todas as threads terminem uma etapa antes da seguinte e observem a mesma condição de parada.

3.5 Uso de SIMD e diagnóstico do compilador

As duas reduções sobre células usam `simd`: a redução de estatísticas a cada passo e a contagem linear dos cinco estados finais. A diretiva declara ao compilador que as iterações não possuem dependências que impeçam a vetorização, mas não garante por si só a emissão de instruções vetoriais. Por isso, a versão final também foi compilada no cluster com o relatório de otimizações do GCC 5.3.1. O compilador registrou explicitamente que o laço linear da contagem final foi vetorizado; não emitiu a mesma confirmação para a redução bidimensional por passo.

O laço principal de atualização preserva o cálculo dos oito vizinhos de Moore em sua forma comum às versões sequencial e paralela. Vetorizá-lo exigiria uma reescrita mais ampla, com tratamento separado das bordas e desenrolamento dos vizinhos. Essa

variante pode ser avaliada em um experimento futuro, mas não foi misturada à solução final sem uma validação e uma comparação controladas próprias.

3.6 Cláusulas de escopo e `default(none)`

As dimensões, parâmetros, matrizes e contadores são compartilhados. O passo é inicializado com `firstprivate`, e os índices dos laços são privados por definição. O uso de `default(none)` torna essa classificação explícita e reduz o risco de compartilhamento incorreto.

3.7 Troca segura das matrizes e condição de parada

As matrizes são alternadas por troca de ponteiros (*double buffering*), evitando cópias completas ao final de cada passo. A troca e a atualização de `passos_executados` ocorrem em uma região `single`. A barreira implícita ao final dessa região garante que nenhuma thread inicie o passo seguinte com ponteiros antigos. A condição de parada utiliza o contador compartilhado produzido pela redução anterior.

3.8 Diagrama da solução

A Figura 1 apresenta o fluxo comum às duas versões e destaca as construções usadas pela versão OpenMP. As barreiras indicadas são implícitas ao final das regiões de compartilhamento de trabalho e da região `single`.

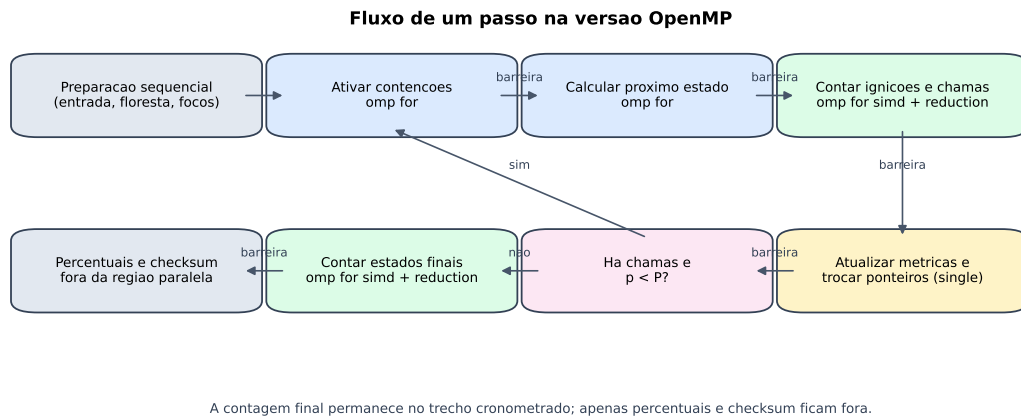


Figura 1: Fluxo geral das versões sequencial e paralela.

3.9 Comparação de *schedules*

Foram avaliados `static`, `dynamic` e `guided`, com diferentes tamanhos de bloco. Os resultados e a justificativa da configuração mantida no programa final são apresentados na Seção 6.3.

4. Validação

Esta seção verifica se as versões sequencial e paralela produzem exatamente o mesmo resultado. A comparação considera as saídas completas, os índices de verificação e diferentes quantidades de threads.

4.1 Metodologia de comparação

Cada execução paralela foi comparada automaticamente com uma execução sequencial da mesma entrada, ignorando somente o campo de tempo. O orquestrador interrompe a campanha caso qualquer outro campo seja diferente. Também foram verificadas, após cada execução, a soma das quantidades finais de células e a igualdade entre células atingidas pelo fogo e focos iniciais mais novas ignições.

Além das três entradas oficiais, foi usado um oráculo independente para 80 grades pequenas, incluindo dimensões degeneradas, $P = 0$, diferentes ventos, limiares, sementes, focos e zonas sobrepostas. Cada caso foi comparado com a versão sequencial e com a versão OpenMP usando 1, 2, 3, 4 e 8 threads e três *schedules*, totalizando 1.280 execuções válidas. Foram verificadas ainda 36 execuções de entradas inválidas.

4.2 Casos de teste

Os resultados resumidos na Tabela 1 confirmam a equivalência das duas implementações nas três cargas avaliadas.

Tabela 1: Validação das versões sequencial e paralela.

Entrada	Dimensão	Passos	Threads	<i>Checksum</i>	Resultado
Pequena	400×500	80	4	17073017104646724864	Idêntico
Média	1200×1500	100	8	17187382406529706254	Idêntico
Grande	2500×2500	100	8	14825089465781038537	Idêntico

4.3 Determinismo em relação ao número de threads

Nas grades pequenas, a saída paralela coincidiu com o oráculo e com a versão sequencial para 1, 2, 3, 4 e 8 threads, sob *static*, *dynamic* e *guided*. Nas entradas oficiais, foram testadas 1, 2, 4, 8 e 16 threads. Em todos os casos, os campos funcionais permaneceram idênticos; somente o tempo variou.

5. Ambiente e Metodologia Experimental

Esta seção caracteriza a plataforma de testes e o procedimento usado para obter as medições. São apresentados o hardware, o software, as cargas, as repetições e as métricas

adotadas.

5.1 Hardware e software

A Tabela 2 registra a configuração do nó `hal01`, no qual todas as medições apresentadas neste relatório foram executadas. Não foram combinados tempos de máquinas diferentes.

Tabela 2: Ambiente utilizado nos experimentos.

Item	Especificação
Processador	Intel Core i7-4790, 3,60 GHz
Núcleos / processadores lógicos	4 / 8
Memória RAM	31 GiB
Sistema operacional	Ubuntu 24.04.4 LTS, Linux 5.15.75, x86_64
Compilador	GCC 5.3.1
Flags	<code>-Wall -Wextra -O2 -fopenmp</code>

5.2 Parâmetros das entradas de teste

As cargas usam matrizes de 400×500 , 1200×1500 e 2500×2500 , com 80, 100 e 100 passos máximos, respectivamente. Elas incluem diferentes sementes, direções e intensidades de vento, focos iniciais e zonas de contenção, permitindo avaliar o algoritmo em escalas distintas.

5.3 Metodologia de medição

Um script executou toda a campanha de forma serial: uma configuração por vez, no mesmo nó e com os mesmos binários. Foi feita uma execução de aquecimento seguida de cinco execuções medidas. A ordem das configurações foi embaralhada em cada repetição medida para reduzir viés de ordem. Os gráficos apresentam a média e, para os tempos, barras com intervalo de confiança de 95% calculado com a distribuição t de Student.

Cada campanha é armazenada em um diretório próprio. O orquestrador recusa iniciar caso encontre resultados existentes e cria logs e manifestos identificados por horário UTC e hash do fonte, impedindo a substituição silenciosa de uma execução anterior. As 132 execuções da campanha de desempenho e as 1.316 execuções da validação funcional foram iniciadas pelos respectivos scripts, sem comandos manuais por configuração.

O tempo foi medido por `omp_get_wtime()` somente no núcleo da simulação: ativação, propagação, estatísticas, troca de matrizes e condição de parada. Leitura, alocação, inicialização, *checksum*, percentuais e impressão ficaram fora da medição. O ambiente definiu `OMP_DYNAMIC=FALSE`, `OMP_PROC_BIND=FALSE` e `OMP_WAIT_POLICY=PASSIVE` em toda a campanha.

Foram utilizadas as entradas pequena, média e grande fornecidas no projeto. Os testes de escalabilidade empregaram 1, 2, 4, 8 e 16 threads. Assim, 4 threads correspondem aos núcleos físicos, 8 utilizam os dois processadores lógicos de cada núcleo e 16 caracterizam a sobresubscrição. O *speedup* e a eficiência foram calculados por

$$S(T) = \frac{t_{seq}}{t_{omp}(T)} \quad \text{e} \quad E(T) = \frac{S(T)}{T}.$$

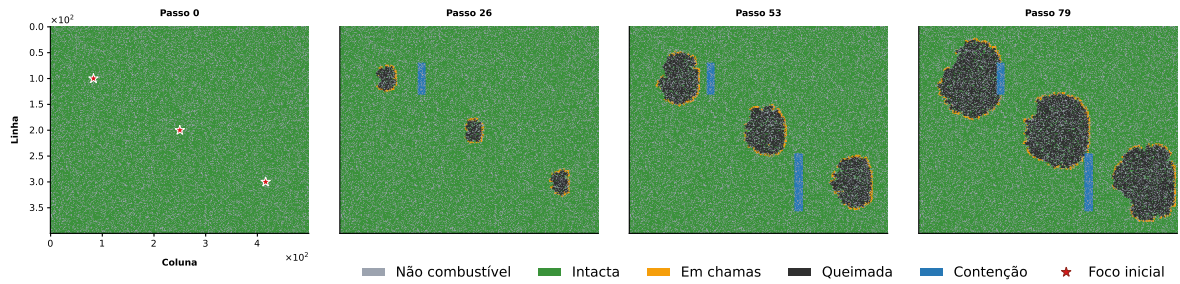
6. Resultados

Esta seção reúne os resultados funcionais e de desempenho. Primeiro é mostrado o comportamento da simulação e, em seguida, são comparados tempo, aceleração, eficiência e *schedules*.

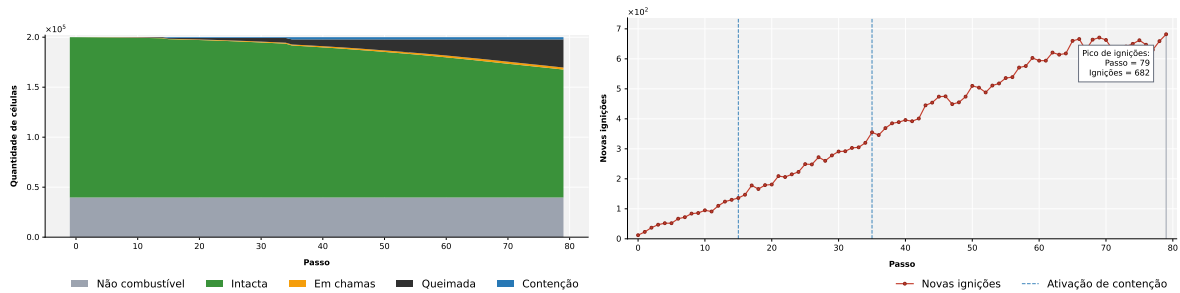
6.1 Visualização do comportamento das entradas

Para cada carga foram observados três aspectos: a distribuição espacial dos estados, a quantidade de células em cada estado e as novas ignições por passo. As figuras foram produzidas por um visualizador independente que reproduz as regras da simulação e foi conferido contra as métricas e os *checksums* dos programas em C. Como o modelo é determinístico e as saídas funcionais das versões sequencial e OpenMP são idênticas, uma única visualização representa ambas; a diferença entre elas é avaliada pelo tempo.

Entrada pequena



(a) Distribuição espacial dos estados.



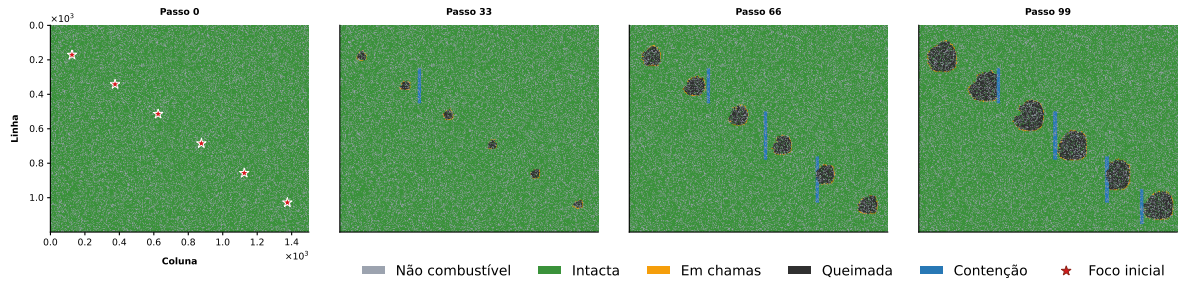
(b) Evolução dos estados.

(c) Novas ignições por passo.

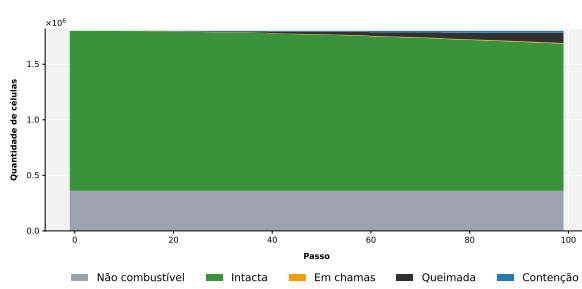
Figura 2: Comportamento determinístico da entrada pequena.

Na Figura 2, as três frentes permanecem separadas durante os 80 passos. A distribuição espacial da Figura 2a mostra o bloqueio parcial causado pelas contenções dos passos 15 e 35; a Figura 2b mostra o crescimento gradual da área queimada, e a Figura 2c registra o pico de 682 ignições no passo 79.

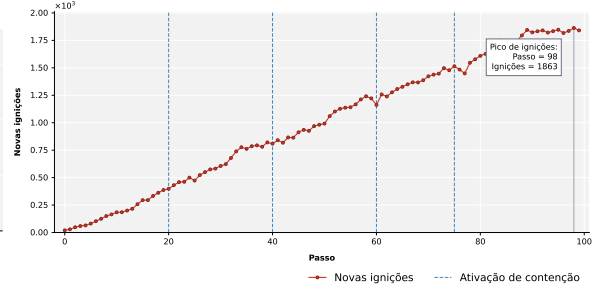
Entrada média



(a) Distribuição espacial dos estados.



(b) Evolução dos estados.

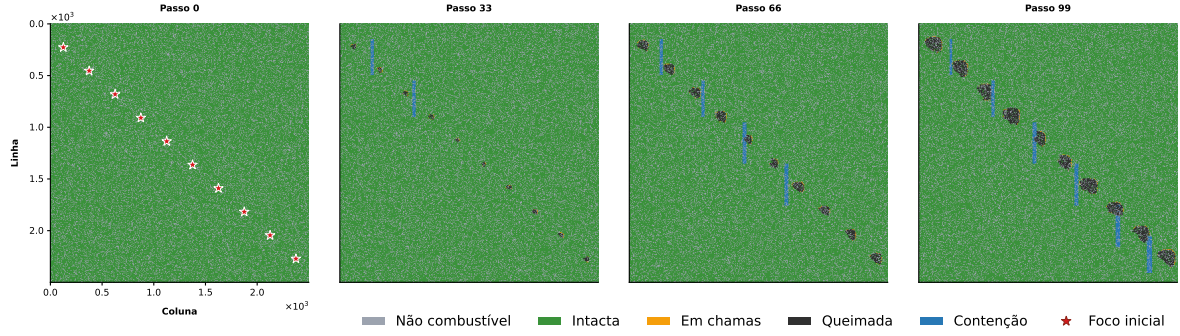


(c) Novas ignições por passo.

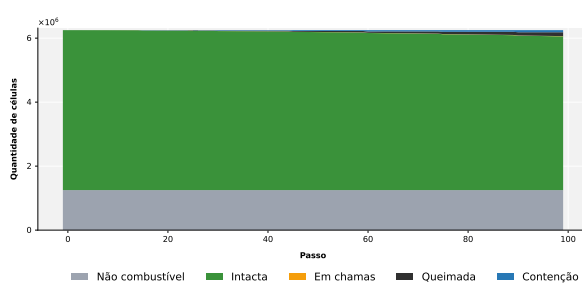
Figura 3: Comportamento determinístico da entrada média.

Na Figura 3, os seis focos originam frentes maiores e as quatro zonas de contenção limitam regiões distintas. A Figura 3a evidencia essa distribuição; a Figura 3b indica que ainda há células em chamas no limite de 100 passos, e a Figura 3c mostra o pico de 1.863 ignições no passo 98.

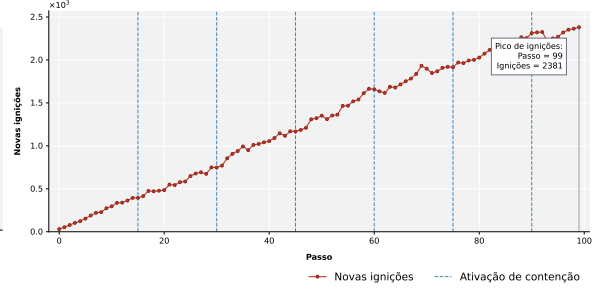
Entrada grande



(a) Distribuição espacial dos estados.



(b) Evolução dos estados.



(c) Novas ignições por passo.

Figura 4: Comportamento determinístico da entrada grande.

Na Figura 4, os dez focos distribuem a propagação por uma área maior. As seis barreiras são visíveis na Figura 4a; a transição dos estados aparece na Figura 4b, e a Figura 4c registra crescimento até o pico de 2.381 ignições no passo 99. Nas três cargas, o aumento das ignições decorre da expansão das frentes, e não de divergência entre as implementações.

6.2 Tempo, *speedup* e eficiência

A Tabela 3 apresenta as médias de cinco execuções medidas. O valor após $\pm$ é a semi-amplitude do intervalo de confiança de 95%. A Figura 5 reúne as métricas derivadas desses tempos e a comparação de escalonamento.

Tabela 3: Tempos médios e intervalos de confiança de 95%, em segundos.

Configuração	Pequena	Média	Grande
Sequencial	0,420 $\pm$ 0,010	4,757 $\pm$ 0,031	16,546 $\pm$ 0,010
OpenMP, 1 thread	0,450 $\pm$ 0,009	5,099 $\pm$ 0,028	17,742 $\pm$ 0,017
OpenMP, 2 threads	0,229 $\pm$ 0,009	2,616 $\pm$ 0,024	9,013 $\pm$ 0,024
OpenMP, 4 threads	0,126 $\pm$ 0,006	1,383 $\pm$ 0,015	4,795 $\pm$ 0,020
OpenMP, 8 threads	0,109 $\pm$ 0,005	1,183 $\pm$ 0,004	4,131 $\pm$ 0,013
OpenMP, 16 threads	0,146 $\pm$ 0,004	1,202 $\pm$ 0,006	4,175 $\pm$ 0,036

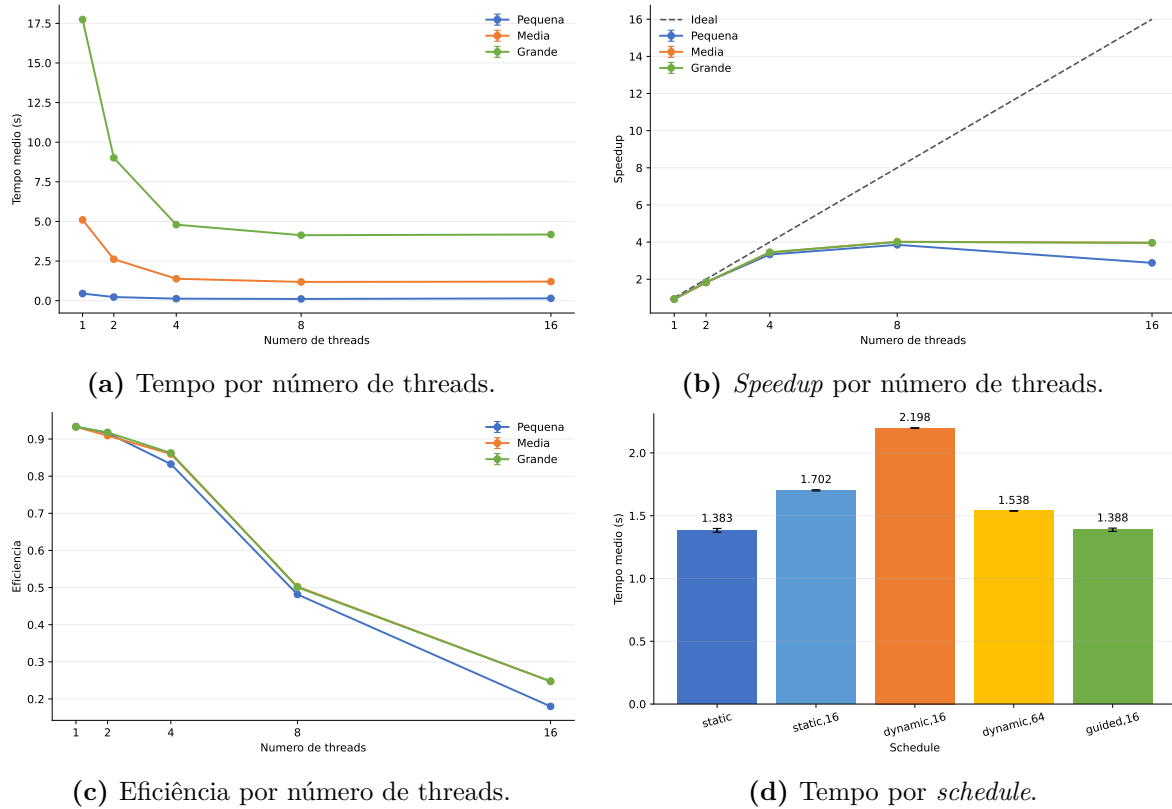


Figura 5: Desempenho da versão OpenMP e comparação das políticas de escalonamento.

O tempo diminui até 8 threads nas três cargas, como mostra a Figura 5a. A Figura 5b evidencia ganho sublinear: o maior *speedup* medido foi 4,02, na carga média com 8 threads. A Figura 5c mostra eficiência entre 83,3% e 86,3% com quatro threads; ao usar as oito CPUs lógicas, a eficiência fica próxima de 50%. Com 16 threads, a sobresubscrição aumenta o tempo nas três cargas.

6.3 Comparação de *schedules*

Os *schedules* foram comparados na entrada média com 4 threads, correspondentes aos quatro núcleos físicos. Cada alternativa foi aplicada à ativação, à atualização e à redução de estatísticas. Os valores exatos estão na Tabela 4 e sua representação visual aparece na Figura 5d.

Tabela 4: Comparação de *schedules* com 4 threads.

<i>Schedule</i>	Bloco	Tempo médio $\pm$ IC 95% (s)
static	padrão	1,383 $\pm$ 0,015
static	16	1,702 $\pm$ 0,005
dynamic	16	2,198 $\pm$ 0,003
dynamic	64	1,538 $\pm$ 0,003
guided	16	1,388 $\pm$ 0,013

7. Análise dos Resultados

Esta seção interpreta os resultados apresentados anteriormente. A discussão destaca a escalabilidade, os principais limitadores e a escolha do *schedule* mantido na implementação final.

7.1 Escalabilidade observada

Com uma thread, a versão OpenMP foi de 7,1% a 7,2% mais lenta que a sequencial devido ao custo da região paralela e das sincronizações. A partir de duas threads houve redução consistente do tempo. Com quatro threads, os *speedups* foram 3,33, 3,44 e 3,45 para as entradas pequena, média e grande. O uso das oito CPUs lógicas elevou esses valores para 3,85, 4,02 e 4,01. A configuração com 16 threads não melhorou nenhuma das cargas.

7.2 Gargalos identificados

A eficiência diminuiu com o aumento de T : na carga grande, foi 91,8% com duas threads, 86,3% com quatro, 50,1% com oito e 24,8% com 16. Até quatro threads, cada thread pode ocupar um núcleo físico; de quatro para oito, o ganho adicional vem do *simultaneous multithreading* e é menor. Com 16 threads há sobresubscrição das oito CPUs lógicas. Além disso, as barreiras entre etapas, a leitura repetida das matrizes e o custo serial da região **single** limitam a escalabilidade. A geração da floresta e o *checksum* são seriais, mas ficam fora do núcleo cronometrado conforme exigido.

7.3 Justificativa do *schedule* escolhido

Static sem bloco explícito e **guided,16** produziram médias praticamente iguais, 1,383 s e 1,388 s, com intervalos de confiança sobrepostos. Portanto, a diferença observada entre elas não sustenta afirmar que uma seja mais rápida. **Dynamic,64** foi 11,2% mais lento; **static,16**, 23,1%; e **dynamic,16**, 58,9%. A implementação final usa **static** como padrão por ser simples, determinístico na distribuição e compatível com o trabalho aproximadamente regular por célula, mantendo **schedule(runtime)** para permitir a comparação experimental.

8. Conclusão

Esta seção sintetiza a correção e o desempenho alcançados pelas duas implementações. Também registra as limitações observadas e possibilidades diretas de continuidade experimental.

As versões sequencial e OpenMP implementaram as mesmas regras e produziram resultados idênticos para todas as entradas avaliadas e para diferentes números de

threads. A região paralela persistente, a divisão dos laços com `omp for`, o uso de `simd`, as reduções e a troca protegida das matrizes permitiram paralelizar o núcleo sem condições de corrida observadas.

O melhor resultado entre as configurações principais ocorreu com 8 threads na carga média, com *speedup* de 4,02; na carga grande, o valor foi 4,01. A queda de eficiência após os quatro núcleos físicos e a piora com 16 threads mostram que sincronização, acesso à memória, *simultaneous multithreading* e sobresubscrição limitam a escalabilidade. Ainda assim, a execução paralela manteve vantagem clara a partir de duas threads. A comparação de *schedules* mostrou também que blocos pequenos e distribuição dinâmica podem acrescentar custo sem compensação para esta carga.

Como continuidade, podem ser avaliados mais números de repetições, afinidade de threads e outras estratégias de vetorização e balanceamento. Essas possibilidades não alteram a solução exigida, mas podem ajudar a caracterizar melhor os limites de desempenho.